Five-rule iPRA with a single evaluation schema

Clause contexts, zero normalization, and a smaller complete verifier

Prepared for Vladimir Reshetnikov

November 11, 2026

Abstract

We give a complete presentation of full first-order intuitionistic primitive recursive arithmetic with two formula constructors, equality and a paired binder, and five inference patterns: reflexivity, one uniform evaluation schema, clause use, paired elimination, and paired introduction. Zero iteration is contracted during term construction and substitution; successor iteration is retained as syntax. The evaluation schema performs both computation and all equality reasoning, with no separate equality-only context parameter and no case split. The presentation is bi-interpretable and equiconsistent with iPRA, with whole numerical domains, actual equality, and numerical composite graph $x = y$.

The construction combines two existing reductions in the current repository: single successor evaluation on zero-normal terms and clause contexts that remove first-class conditional closure. A shorter two-evaluation congruence proof improves the accompanying proof compiler. The complete verifier is measured against controls with the same input discipline, including parsing, validation, normalization, substitution, and inference. The reduction is strict in all three source-size coordinates against the matched raw control. It does not establish an absolute minimum or beat the repository's separate 516-byte compact codec. The new equivalence is a written proof, with executable proof translations and independent implementation checks; it is not a new Lean or Rocq formalization.

Contents

1 Result and comparison boundary **4**
1.1 What is being improved . . . 4
1.2 Measured complete checkers . . . 5

2 The entire formal theory **5**
2.1 Normal terms and substitution . . . 5
2.2 Formulas, retained clauses, and falsity . . . 6
2.3 All five primitive inference patterns . . . 7

3 Recovering equality and the missing case introduction **7**

4 Exact consequence equivalence **9**
4.1 Conditional closure is a macro . . . 9
4.2 Comparison with the zero-normal native kernel . . . 9

5 Full iPRA and both interpretation composites **11**
5.1 The ordinary intermediate theory . . . 11
5.2 Explicit logical macros . . . 11
5.3 The internal arithmetic bridge . . . 12
5.4 Why the result is a bi-interpretation . . . 13

6 Complete implementation and verification **13**
6.1 Public contract and trusted boundary . . . 13
6.2 Executable evidence . . . 14
6.3 Reproduction and limits . . . 14

Attribution and inherited foundations **16**

A Complementary introductions with one local assumption **16**
A.1 Complete native syntax and rule inventory . . . 16
A.2 Exact equivalence and the unused-witness construction . . . 17
A.3 Induction and numerical cases separate cleanly . . . 18
A.4 The entire ordinary logical interface . . . 19
A.5 Whole-domain realizations and both formula composites . . . 20
A.6 What the theorem does and does not claim . . . 21

B Arithmetic strength and both ordinary interpretation composites **21**
B.1 The arithmetic dependency order . . . 22

C Internal scalar predecessor bootstrap **23**
C.1 Elementary terms and two finite cycles . . . 23
C.2 A ramp with a global scaling identity . . . 24
C.3 A comparator from scalar iteration . . . 25
C.4 Saturated iteration gives predecessor . . . 25
C.5 Dependency boundary . . . 27

D From literal-equation induction to full intuitionistic PRA **27**
D.1 The exact hypotheses . . . 27
D.2 Constructor separation and two elementary tests . . . 28
D.3 Four elementary uses of equation induction . . . 29
D.4 An equation invariant that reflects actual equality . . . 30
D.5 Compilation of all quantifier-free formulas . . . 31
D.6 Recovery of induction and the full theory . . . 32

E Pairing, primitive recursion, and internal numerical composites **32**
E.1 Pairing from a bounded square-root search . . . 32
E.2 Compiling each primitive recursion . . . 33
E.3 Why both numerical round trips are internal . . . 34

F The complete selected proof checker **35**

1 Result and comparison boundary

The recommended presentation is $\mathcal{C}_5$. Its five primitive inference patterns are

Refl,	Eval,	Use,	Elim,	Intro.

The only formula constructors are $s = t$ and $\mathsf{D}_{x,y}B$. There is one numerical sort. Its terms are variables, 0, successor, and scalar iteration, subject only to the zero normalization specified below. The single evaluation schema is

$$\frac{\Gamma \vdash \mathcal{I}(n, a; u.h) = t \qquad \Gamma \vdash I(Sn, a; u.h) = r}{\Gamma \vdash r = h\langle t/u\rangle}. \tag{Eval}$$

Here $\mathcal{I}(0, a; u.h) = a$ by syntax, and angle brackets denote capture-avoiding substitution followed by zero normalization. Nothing else is evaluated by the term representation. In particular, $I(S0, a; u.h)$ remains an iterator node. This surviving one-step iterator supplies the complete equality interface.

Theorem 1.1 (Main result)**.** *The ordinary realization of $\mathcal{C}_5$ is bi-interpretable and equiconsistent with full first-order* iPRA*. Both interpretations use the whole numerical domain and actual equality. Both numerical composites are internally isomorphic to the identity by the graph $x = y$. The formula translations in both directions have provable round trips. The result holds for arbitrary finite retained contexts of ordinary formulas, not only closed theorems.*

1.1 What is being improved

The comparison is pinned to ProveIt commit `1eb151b8f40800aec569361e827d934cb77bd3e1`. Its index describes 52 filed packages, and its incoming directory contains four more archives. The incoming packages are relevant: ignoring them would overstate novelty. The closest proposals are the following.

Presentation	Formula syntax	Arithmetic/equality interface
Raw clausal CE5	Eq/D	Refl and three-case Eval; ordinary substitution
Zero-normal $\mathcal{Z}_1$	Eq/Q/D	Refl and one successor Eval; six patterns
Canonical clausal	Eq/D	Successor-seeded recursor, computation and supplied transport template
Present $\mathcal{C}_5$	Eq/D	Refl and one successor Eval; five patterns

The mathematical component reductions are inherited and are credited to their repository packages [3, 2, 4, 1]. The contribution here is their exact five-pattern combination, the explicit comparison proof on normal syntax, the shorter congruence compiler, and a complete controlled implementation. No previously known component is presented as a new axiom-deletion theorem. The complete single-evaluation clause combination was not present in the inspected current specifications; this is a comparison with that snapshot, not a literature-priority claim.

The improvement removes actual kernel work: the raw evaluator's root, zero-iteration, and successor-iteration alternatives become one schema. It also removes the native Q formula constructor and its dedicated hypothesis/elimination interface. The normalization traversal and clause representation are explicitly counted. Five patterns alone are not a minimality measure: combining unrelated rules under one name would prove nothing about simplicity.

1.2 Measured complete checkers

The selected standalone program has 2,283 normalized UTF-8 bytes, 741 significant tokens, and 826 AST nodes. Its matched raw control has 2,416 bytes, 744 tokens, and 842 nodes. Thus the byte reduction is 133 bytes (5.50%); the token and AST reductions are 3 and 16.

Complete goal-directed program	Bytes	Tokens	AST nodes	Gzip-9
Raw CE5 control	2,416	744	842	855
Zero-normal control with root case	2,426	765	865	862
Single-Eval clause checker	**2,283**	**741**	**826**	**818**

All entries count the entire acceptance module after removing its module docstring, applying CPython 3.12.14 `ast.unparse`, and appending one LF. Significant tokens exclude whitespace, indentation, comments, and the end marker. AST nodes are counted by `ast.walk`. Gzip uses level 9 and zero timestamp and is informational, not an optimization objective.

The readable selected source is 2,717 bytes, or 1,036 bytes with gzip-9. The readable raw control is 2,698 bytes, or 984 with gzip-9; thus the normalized-source comparison is not a claim that the original formatted file is shorter. The auxiliary forward implementation is 3,022 normalized bytes, 867 tokens, and 1,008 AST nodes, against 3,046, 853, and 993 for its forward control. It improves bytes only; it is not the selected coordinatewise result.

The main comparison uses the same goal-directed protocol, clause contexts, variable representation, validated reflexive leaves, and public output contract. The successor-only rule naturally supplies the parameters of its two premises, rather than testing several possible shapes after both proofs have been checked. That certificate representation is part of the specified presentation and is included in the measured program.

The historical compact codec has a different representation and a highly compressed proof protocol. The present complete checker is larger than its 516-byte normalized source. Thus this report establishes a structural and matched implementation improvement; it does not assert a new global lower envelope. There is also no proof that a globally simplest axiomatization exists for an unspecified implementation language and coding convention.

2 The entire formal theory

2.1 Normal terms and substitution

Start with the raw syntax

$$t ::= x \mid 0 \mid S(t) \mid I(n, a; u.h).$$

Only u in the update body h is bound. The counter and initializer are outside that binding. Updates may have any fixed numerical parameters. There is no variable for a program or function description and no universal evaluator. Define nf recursively on this finite syntax:

$$\mathrm{nf}(x) = x, \qquad \mathrm{nf}(0) = 0, \quad \mathrm{nf}(St) = S\,\mathrm{nf}(t),$$

$$\mathrm{nf}(I(n, a; u.h)) = \begin{cases} \mathrm{nf}(a), & \mathrm{nf}(n) \equiv 0, \\ I(\mathrm{nf}(n), \mathrm{nf}(a); u.\,\mathrm{nf}(h)), & \text{otherwise.} \end{cases}$$

Bound names are first renamed when needed. A term of $\mathcal{C}_5$ is a fixed point of nf: every iterator node has a counter that is not literally 0. On normal inputs write

$$\mathcal{I}(n, a; u.h) = \begin{cases} a, & n \equiv 0, \\ I(n, a; u.h), & n \not\equiv 0. \end{cases}$$

The normalization is syntactic. A counter provably equal to zero is not contracted unless it is the literal zero after the recursive normalization. Normal substitution is

$$t\langle s/x\rangle := \mathrm{nf}(t[s/x]).$$

Formulas and clauses use the corresponding operation on every contained term. The full successor computation is deliberately not made definitional.

Lemma 2.1 (Zero normalization)**.** *Zero normalization terminates, preserves scope, and is idempotent. It commutes with capture-avoiding substitution in the form*

$$\mathrm{nf}(t[s/x]) = \mathrm{nf}\big(\mathrm{nf}(t)[\mathrm{nf}(s)/x]\big).$$

Normal substitutions obey the usual composition identities. For each raw term, the ordinary iterator theory proves $t = \mathrm{nf}(t)$*.*

Proof. The definition recurses only on proper subterms and contracts an entire zero-counter iterator to its already normalized initializer. Induction on syntax proves the first three assertions and the displayed identity. The iterator case distinguishes whether its normalized substituted counter is literal zero; both sides then make the same choice. The ordinary substitution composition law followed by this identity gives the normal version.

For provable equality, use the zero computation equation and congruence on the counter and seed. For changes inside the update body, equation induction proves extensionality of a fixed iterator: equal seeds and pointwise equal updates give equal iterates. This handles the binder body without treating it as an unmentioned ordinary function argument. The repository's normalization development proves this comparison formally for the reference kernel [5]. □

2.2 Formulas, retained clauses, and falsity

The complete formula grammar is

$$A ::= (s = t) \mid \mathsf{D}_{x,y}A.$$

The paired constructor binds two distinct numerical variables. A context is a finite reusable list of clause occurrences

$$[A; y.B(y)], \qquad y \notin \mathrm{FV}(A).$$

The clause binds y only in its consequent. It is an object of the context, not a third formula constructor. An ordinary assumption F abbreviates

$$\mathrm{ord}(F) := [H; v.F], \qquad H := (0 = 0), \quad v \notin \mathrm{FV}(F).$$

Accordingly, Γ, F in a rule below means $\Gamma, \mathrm{ord}(F)$. Clause occurrences are distinguished, even when two occurrences have identical text. The designated contradiction is $0 = S0$.

2.3 All five primitive inference patterns

All numerical data and formulas must be well scoped and normal. Every instance uses the normal substitution just defined.

$$\frac{}{\Gamma \vdash s = s}. \tag{Refl}$$

$$\frac{\Gamma \vdash \mathcal{I}(n, a; u.h) = t \qquad \Gamma \vdash I(Sn, a; u.h) = r}{\Gamma \vdash r = h\langle t/u\rangle}. \tag{Eval}$$

$$\frac{\Gamma \vdash A}{\Gamma \vdash B\langle t/y\rangle} \quad \text{when } [A; y.B] \text{ is retained in } \Gamma. \tag{Use}$$

$$\frac{\Gamma \vdash \mathsf{D}_{x,y}B(x, y) \qquad \Gamma, [B(b, 0); y.B(b, y)] \vdash F}{\Gamma \vdash F}. \tag{Elim}$$

$$\frac{\Gamma, B(t, \eta) \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y}B(x, y)}, \qquad \eta = \begin{cases} z, & B \text{ is a literal equation and } t \equiv 0, \\ 0, & \text{otherwise.} \end{cases} \tag{Intro}$$

In Elim, b is fresh for Γ, F, and all passive parameters of B. In Intro, z is fresh for Γ, t, and all passive parameters of B. The displayed local clause or ordinary assumption is discharged. The update variable u in Eval binds only h. No freshness condition excludes passive parameters from that update.

There are no further logical, structural, equality, successor, computation, or induction axioms or rules. In particular, there is no primitive hypothesis rule, no Q formula constructor, no arbitrary formula induction, and no separate equality-transport rule. Eval does supply the iterator update h, whose occurrences are checked in its computation premises; there is no additional equality-only context parameter. The mode test in Intro is part of the specification, not an unstated inference choice.

3 Recovering equality and the missing case introduction

Lemma 3.1 (Elementary structural operations)**.** *Ordinary hypothesis use, weakening, injective fresh renaming, and cut for ordinary formula assumptions are admissible in $\mathcal{C}_5$.*

Proof. Refl proves H, and Use on $[H; v.F]$ proves F. For weakening and renaming, induct on a derivation, first freshening eigenvariables away from new retained data. Injective renaming preserves normal terms, equation roots, and literal zero, so it preserves Intro's mode.

For cut, replace a Use of the distinguished occurrence $[H; v.F]$ by the supplied proof of F, weakened into its local context. At other rules transform the premises recursively. The extra proof of H at that Use can be discarded. No arbitrary numerical substitution into a proof is assumed in this argument. □

Lemma 3.2 (Identity frame and equality composition)**.** *From $p : s = t$ and $q : s = r$, $\mathcal{C}_5$ derives $r = t$ using two Eval nodes and one new reflexivity leaf, with each supplied proof used once.*

Proof. Let $K(s) = I(S0, s; u.u)$. Its predecessor iterate is definitionally $\mathcal{I}(0, s; u.u) = s$. Thus

$$\mathrm{Root}(p, q) := \mathsf{Eval}\big(p, \mathsf{Eval}(q, \mathsf{Refl}(K(s)))\big) : r = t.$$

The inner Eval proves $K(s) = r$, and the outer one proves $r = t$. No equality law or logical principle is used beforehand. □

Symmetry is $\mathrm{Root}(\mathsf{Refl}(s), p) : t = s$ for $p : s = t$. Transitivity of $p : s = t$ and $q : t = v$ is $\mathrm{Root}(q, \mathrm{Sym}(p)) : s = v$. Successor computation is a single Eval on two reflexivities:

$$I(Sn, a; u.h) = h\langle \mathcal{I}(n, a; u.h)/u \rangle.$$

Zero computation is already an identity of the normal syntax.

Lemma 3.3 (Congruence in two evaluations)**.** *For every normal numerical template $h(u)$, from $p : s = t$ one can derive $h\langle s/u\rangle = h\langle t/u\rangle$ by two Eval nodes and two reflexivity leaves. The input proof occurs once.*

Proof. Put $J = I(S0, s; u.h)$. First construct

$$e_s := \mathsf{Eval}(\mathsf{Refl}(s), \mathsf{Refl}(J)) : J = h\langle s/u\rangle.$$

The left side of this equation is still the successor iterator J. Consequently a second application,

$$\mathsf{Eval}(p, e_s) : h\langle s/u\rangle = h\langle t/u\rangle,$$

is an instance of exactly the same schema. The first premise now uses the supplied equality in place of the reflexivity of the seed. This avoids constructing two separately evaluated endpoints and joining them afterwards. □

Corollary 3.4 (Template transport and exact local overhead)**.** *From $p : s = t$ and $q : h\langle s/u\rangle = r$, derive $r = h\langle t/u\rangle$ with four Eval nodes and three reflexivity leaves. Each of p, q occurs once. Replacing k such one-term transport nodes in a proof whose other rules are retained adds exactly $6k$ inference nodes.*

Proof. Apply Root to the congruence of the preceding lemma and q. The replacement has seven new nodes in place of the old one, and it does not duplicate either premise tree. This counts inference nodes, not the serialized sizes of repeated term labels. □

Transport in both sides of an equation follows from two crossed one-term transports: $a(s) = b(s)$ first gives $b(s) = a(t)$, then $a(t) = b(t)$. No tree-size bound that hides the two uses of the equality proof is claimed. The shorter congruence construction is also used by the executable certificate compiler.

Lemma 3.5 (Unrestricted case introduction)**.** *For every matrix B and every normal witness t,*

$$\frac{\Gamma, B(t, 0) \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y} B(x, y)}$$

is admissible, with the usual freshness condition on z.

Proof. Only a literal equation matrix at witness 0 is excluded by the primitive case mode. In that situation set

$$Z = I(S0, 0; u.0).$$

This is a valid normal term, is not literally zero, and Eval proves $Z = 0$. Under the local equation $B(Z, 0)$, use equation-template transport to derive $B(0, 0)$. Cut in the supplied case proof to obtain $B(0, Sz)$, then use the reverse equality to derive $B(Z, Sz)$. Intro at witness Z is in case mode and has the desired conclusion $D_{x,y}B(x, y)$. Its witnessing term has changed; its conclusion has not.

The raw alias $I(0, 0; u.0)$ cannot be used here: it normalizes to 0. The one-step alias Z survives because successor computation is not a normalization rule. This is the relevant interaction between the two component reductions. □

Lemma 3.6 (Transport through all formulas)**.** *For every Eq/D formula template $F(v)$, $s = t$ and $F(s)$ imply $F(t)$ in the same retained context.*

Proof. Use external structural induction on F. The equation case was proved above. For $F(v) = D_{x,y}B(v, x, y)$, eliminate $F(s)$ at fresh b, opening $[B(s, b, 0); y.B(s, b, y)]$. Introduce $F(t)$ by the unrestricted case rule with witness b. Convert its local base $B(t, b, 0)$ to $B(s, b, 0)$ by the induction hypothesis and symmetry. Use the opened clause at Sz and convert forward to $B(t, b, Sz)$. Close introduction and elimination. Every recursive transport concerns the smaller matrix. □

4 Exact consequence equivalence

4.1 Conditional closure is a macro

For an unused fresh numerical variable w, put

$$q_y B := D_{w,y}B.$$

This has the ordinary reading $B(0) \to \forall y B(y)$, since the dummy existential witness ranges over an inhabited numerical domain.

Lemma 4.1 (Closure interfaces)**.** *The following three constructions are admissible:*

1. *From $\Gamma, B(0) \vdash B(Sz)$, derive $\Gamma \vdash q_y B$.*
2. *From $\Gamma \vdash q_y B$ and $\Gamma \vdash B(0)$, derive $\Gamma \vdash B(t)$ for every normal t.*
3. *In a context containing $[B(0); y.B(y)]$, derive $q_y B$.*

For an equation $E(y)$, one also derives $q_y E$ from $\Gamma, E(z) \vdash E(Sz)$.

Proof. The first construction is case introduction with an unused witness. For the second, eliminate the dummy existential and Use the resulting clause with the supplied base. The opened witness does not occur in B. For the third, use case introduction and specialize the retained clause at Sz in its branch. For equation progress, use primitive Intro with dummy witness 0; it selects the progress assumption $E(z)$. □

4.2 Comparison with the zero-normal native kernel

Let $\mathcal{N}_0$ denote the repository's zero-normal kernel, `NZPrf`. It has Eq/Q/D formulas and ordinary formula contexts. Its rules are Hyp, successor computation, crossed one-term transport, Q elimination, D elimination, unrestricted case introduction, and equation-progress introduction of Q. The mathematical comparison is made with that kernel, rather than assuming that witness-selected Intro is preserved directly by all numerical substitutions.

The Lean encoding permits raw numerical rule parameters in some places. Normalize those parameters first. The normalization/substitution identities preserve the resulting normal premises and conclusion, so the canonical-data presentation used here has exactly its normal sequents.

Define T on its formulas by retaining equality and D, and replacing every $Q_y B$ by $D_{w,y}T(B)$ with a fresh unused w. Let U include canonical Eq/D formulas into the native formula grammar. Numerical terms are unchanged by these two maps.

Theorem 4.2 (Exact normal-sequent comparison)**.** *For every finite normal native context Γ and normal native formula A,*

$$\Gamma \vdash_{\mathcal{N}_0} A \quad \Longrightarrow \quad \mathrm{ord}(T\Gamma) \vdash_{\mathcal{C}_5} T(A).$$

For normal Eq/D contexts Δ and normal formulas F,

$$\mathrm{ord}(\Delta) \vdash_{\mathcal{C}_5} F \quad \Longleftrightarrow \quad U\Delta \vdash_{\mathcal{N}_0} UF.$$

Both maps are effective transformations of finite proof trees.

Proof. Translate a native proof recursively. Hyp is Refl at 0 followed by Use. Successor computation and crossed transport are derived above. The two native introductions are supplied by the case lemma and the progress clause in Lemma 4.1. Native Q elimination uses its second interface.

For native D elimination, translate the major proof and open the corresponding clause. The third interface derives $q_y T(B)(b, y)$ in that clause context, which is the translation of the old Q assumption. Use ordinary-assumption cut to insert this proof into the translated branch, then close Elim. This avoids treating a context clause as if it were already a formula assumption.

Conversely, track the origin of each retained clause occurrence. An ordinary clause $[H; v.G]$ is represented by native assumption UG. A clause introduced by Elim is represented by $Q_y UB(b, y)$. Use of the former is Hyp, while Use of the latter is native Q elimination. Refl is derivable natively. Eval follows by successor computation, transport along its first equality, and the reverse of its second equality. Elim becomes native D elimination.

Intro in case mode is the native case rule. In progress mode the matrix is an equation; native equation progress yields its conditional closure, which provides the branch for native case introduction at the specified witness. All freshness conditions are preserved. Initial ordinary contexts and every locally introduced clause have the stated origin, so induction on the proof gives the converse for all such sequents. □

Corollary 4.3 (Numerical substitution on derivations)**.** *Capture-avoiding normal numerical substitution is admissible for ordinary retained-formula sequents of $\mathcal{C}_5$.*

Proof. Translate into the stable zero-normal native kernel, apply its admissible normal substitution, and translate back. The formula translations commute with normal substitution up to fresh bound renaming. This argument permits a witness to become literal zero without assuming that Intro's mode is preserved rule for rule. It supplies the structural operation used in the subsequent logical macro derivations. □

Arbitrary externally retained clauses have their ordinary reading $A \to \forall y B(y)$. They can be represented by the native implication and universal macros; clause use becomes the corresponding two ordinary eliminations. Thus the optional external clause notation adds no arithmetic or logical strength.

Theorem 4.4 (Formula round trips)**.** *For every normal Eq/D formula F, $TUF = F$ literally up to bound renaming. For every normal native formula A,*

$$A \dashv\vdash_{\mathcal{N}_0} UT(A).$$

For raw native formulas, normalize all terms first. The inclusion back into raw syntax is provably equivalent to the original formula.

Proof. The only nontrivial normal formula step is $Q_y B \dashv\vdash D_{w,y} B$ for unused w. From the left, use case introduction and closure elimination. From the right, D elimination opens exactly $Q_y B$, which closes the branch by Hyp. Pointwise interderivability of matrices lifts through D by elimination, case introduction, backward conversion of its zero assumption, and forward conversion of its successor

goal. It lifts through Q by the same unused-witness argument. External structural induction gives the result.

For the raw statement, compose with the zero-normalization comparison `nzprf_iff_nprf` and the provable normal-form equivalence in the pinned development [5]. Normalization is part of the translation; no unproved substitution theorem for witness-selected Intro is needed. □

5 Full iPRA and both interpretation composites

5.1 The ordinary intermediate theory

The ordinary scalar theory T_I is intuitionistic first-order logic with actual equality, 0, successor, and fixed finite scalar-iterator descriptions. Each update $h(u, \vec{p})$ has defining equations

$$I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u].$$

Its other nonlogical principles are coverage, $\forall x(x = 0 \vee \exists y\, x = Sy)$, separation $0 = S0 \to \bot$, and induction for literal equations with arbitrary retained assumptions and the standard freshness condition. The native kernel is a presentation of this ordinary theory, including its logic; it is not merely an equational fragment [6].

The readings of our new syntax are

$$(s = t)^\circ = (s = t), \qquad (D_{x,y}B)^\circ = \exists x(B^\circ(x, 0) \to \forall y B^\circ(x, y)),$$

$$[A; y.B]^\circ = A^\circ \to \forall y B^\circ(y).$$

All five primitive rules are sound under these readings. In particular, progress Intro is sound because its matrix is literally an equation; case Intro uses numerical coverage. Using progress for an arbitrary compound formula would be an unjustified strengthening.

5.2 Explicit logical macros

For normal terms let $C(v, a, b) = \mathcal{I}(v, a; u.b)$, with u unused in b. Its zero value is definitionally a and its successor value is provably b. Define a gate recursively by

$$G_v(s = t) := (C(v, 0, s) = C(v, 0, t)), \qquad G_v(D_{x,y}B) := D_{x,y}G_vB,$$

freshening bound variables away from v. Then G_0A is a theorem, $G_{Sz}A \dashv\vdash A$, and the padding $D_{u,y}G_yA$ is interderivable with A when u, y are fresh. For the padding converse, eliminate the paired formula, supply its zero-gate theorem, and Use its clause at 1. The gate is essential; an unused D without that gate would make a different formula.

Define the selector by recursion on the maximum binder depth:

$$\begin{aligned} M_v(s = t, r = w) &:= (C(v, s, r) = C(v, t, w)), \\ M_v(D_{x,y}A, D_{x,y}B) &:= D_{x,y}M_v(A, B), \\ M_v(E, D_{x,y}B) &:= D_{x,y}M_v(G_yE, B), \\ M_v(D_{x,y}A, E) &:= D_{x,y}M_v(A, G_yE), \end{aligned}$$

where E is an equation and paired binders are aligned and fresh. Padding and matrix congruence prove $M_0(A, B) \dashv\vdash A$ and $M_{Sz}(A, B) \dashv\vdash B$. Let $1 = S0$, $2 = S1$, and $p(z) = \mathcal{I}(z, 0; u.C(u, 1, 0))$; only the computations $p(1) = 1$ and $p(2) = 0$ are needed here.

With all new binders fresh and the first coordinate of q unused, the full logical macro inventory is

$$\begin{aligned}
\bot_* &:= (0 = 1),\\
A \Rightarrow_* B &:= q_y M_y(A, B),\\
A \wedge_* B &:= q_y G_y M_{p(y)}(A, B),\\
\exists_* x\, A(x) &:= D_{x,y} G_y A(x),\\
A \vee_* B &:= \exists_* v\, M_v(A, B),\\
\forall_* x\, A(x) &:= A(0) \wedge_* q_y A(y).
\end{aligned}$$

These are finite formula translations, not additional kernel rules. Implication introduction uses case introduction; its elimination Uses the opened clause at 1. Existential introduction uses the selected witness and the successor gate; its elimination supplies the true zero gate and Uses at 1. Conjunction projections use the true zero gate and the two instances $1, 2$. Its introduction follows by numerical cases on the selector control. Disjunction uses existential witnesses $0, 1$; numerical cases on its control give the elimination rule. The two universal rules follow from the q interfaces and the conjunction rules.

Finally, under $0 = 1$, transport through $C(v, s, t)$ gives $s = t$ for arbitrary terms. External induction on formulas with case introduction then gives every formula, proving falsity elimination. The inherited development in Appendix A gives these derivations in detail; the normalized rules above supply each ingredient. No excluded middle for quantified formulas is assumed.

5.3 The internal arithmetic bridge

The inherited arithmetic development is included in the appendices, rather than replacing an internal interpretation argument by computations on standard numerals. Its essential dependency order is:

1. Derive a predecessor P with $P0 = 0$ and $P(Sn) = n$ from scalar iteration, equation induction, and numerical cases.
2. Build truncated subtraction and a faithful term characteristic of actual equality; its reflection theorem holds at arbitrary model elements.
3. Compile every fixed quantifier-free intuitionistic formula to a characteristic equation, obtaining the full quantifier-free induction scheme.
4. Construct pairing and projections by bounded search, then compile every fixed primitive recursion to scalar iteration.

The predecessor is established before assuming a pairing function or a faithful equality detector. This ordering is necessary to avoid circularity.

For pairing π with projections L, R, the recursion

$$f(0, \vec{p}) = g(\vec{p}), \qquad f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p})$$

is represented by

$$\begin{aligned}
V(n, \vec{p}) &= I\big(n, \pi(0, g^*(\vec{p})); u.\pi(SL(u), h^*(L(u), R(u), \vec{p}))\big),\\
f^*(n, \vec{p}) &= R(V(n, \vec{p})).
\end{aligned}$$

Equation induction proves $L(V(n, \vec{p})) = n$ and the defining equations for f^*. Each star is a finite syntactic compilation of a fixed description. No total numerical evaluator for all primitive-recursive descriptions is introduced.

5.4 Why the result is a bi-interpretation

Interpret scalar iteration in ordinary iPRA by primitive recursion whose step ignores the recursion counter. Interpret each iPRA function back into T_I by the scalar compiler. The domain formula in each direction is $x = x$, and equality is unchanged. Uniqueness of each fixed recursion follows from equation induction. External induction on the finite description then proves, internally, that each doubly translated function equals its original. This also supplies coherence when substitution changes an update description.

The numerical identity graph $x = y$ is therefore internally a total bijection preserving every operation in both composites. Formula round trips follow from these numerical equalities, the derived intuitionistic logical rules, and the native/D equivalences of Theorem 4.4. In particular the paired formula has the expected internal equivalence with

$$\exists_* x(B(x,0) \Rightarrow_* \forall_* y B(x,y)).$$

The forward direction opens its clause and proves all instances under the base; the reverse direction uses the given universal in case introduction. This proves Theorem 1.1. The translation also identifies $0 = S0$ with ordinary falsity, yielding equiconsistency.

The pinned Lean endpoints `native_iff_pra`, `pra_iff_native`, `native_roundTrip`, `pra_native_roundTrip`, and `native_inconsistent_iff` document the inherited bridge [6]. The new clause/single-Eval comparison is the written proof above. These existing source files were inspected and are included as references; they were not rebuilt in this run.

6 Complete implementation and verification

6.1 Public contract and trusted boundary

The selected checker is a single Python standard-library module. Its input is one JSON object with exactly `goal` and `proof`. The goal must be a closed normal Eq/D formula. Its successful public result is exit status zero and exactly `ACCEPT` followed by a newline. Invalid certificates print `REJECT` and exit with status one.

The numeral 0 denotes zero; positive integers are local bound indices, starting at 1; negative integers are fresh parameter levels. At ambient scope n, the available free parameters are $-1, \ldots, -n$. Successor is `[0,t]`, iteration is `[n,a,h]`, equality is `[s,t]`, and D is `[B]`. An iterator binds one index only in its third entry; D binds two, with y at index 1 and x at index 2. Term and formula validation use the expected sort, so an equality pair cannot silently stand for a successor term.

The five proof patterns are represented as follows. A validated reflexive goal closes immediately; its proof payload is unused, and may be JSON null. Every control uses the same convention. Other nodes use these packets:

Rule	Packet
Eval	`['v',n,a,h,t,major,minor]`
Use	`['u',index,t,premise]`
Elim	`['e',body,major,branch]`
Intro	`['i',witness,branch]`

These are readable Python-style displays of JSON arrays; actual files use double-quoted strings. Eval constructs its predecessor and successor equations from the supplied counter, seed, and update. In a goal $r = w$, it checks $w = h\langle t/u\rangle$, then verifies both prescribed premises. The constructed successor

premise validates all the counter, seed, and update data, including data that a zero predecessor would discard. The major premise validates t. This avoids redundant independent traversals without weakening the validation invariant.

The opening traversal does not shift its inserted value because every replacement is a depth-zero term at the current ambient scope: only 0 and negative free levels occur outside its own binders. It still tracks the binders traversed inside the target. Elim validates the parent goal before admitting a fresh level, preventing eigenvariable escape; Intro validates its witness before opening its fresh step variable.

All syntax validation, normalizing opening, branch scope, clause handling, JSON parsing, and public output are included in the size measurements. The proof compiler, tests, inherited sources, and operating-system/Python runtime are not part of the standalone kernel. They are reported separately. Acceptance does not depend on Python `assert`, so optimization mode does not remove proof checks.

6.2 Executable evidence

The packaged reproduction run passed all of the following:

- 66 raw-to-normal certificate translations, each checked in the raw CE5 verifier, the forward single-Eval verifier, the selected checker, the matched controls, and an independent archived normalized Eq/Q/D verifier after the reverse translation.
- 70 accepted certificates and 20 adversarial rejected certificates.
- 180 public-interface executions, once in ordinary Python and once with `python-O` for each accepted or rejected certificate.

The 66 source certificates comprise 32 original CE5 examples, 32 distinct round-tripped proof certificates for those examples, and two explicit witness-normalization repairs. They are not 66 independent mathematical phenomena. Four additional positive cases exercise a genuinely used induction hypothesis and three closure targets. Rejections include false equations, malformed or incorrectly scoped terms, Boolean or floating-point indices, wrong evaluation premises, illegal witnesses, and an invalid change from progressive to case introduction.

Two independent reviews examined the mathematical comparison and the implementation. The implementation review confirmed the validation-through-premises invariant, the scope discipline, and the harmless unused payload at reflexive leaves. The packaged `evidence/verification.json` records the reproducible run; `evidence/implementation_audit.txt` records the review boundary.

The forward compiler preserves the end formula up to zero normalization. It explicitly repairs a case-introduction witness that normalizes to 0, using the surviving alias Z. A reverse compiler checks produced proofs against an independently supplied normalized native checker with Eq/Q/D formulas. These transformations are untrusted conveniences: the new checker validates their output independently.

The finite checks establish implementation evidence for the supplied examples and adversarial cases. They do not prove the universal mathematical metatheorems or certify the Python implementation in Lean. Proof-tree translation, checker correspondence, and the new complete bi-interpretation remain written mathematical arguments, with the stated independent reviews.

6.3 Reproduction and limits

Run `python3reproduce.py` from the extracted package. It reruns the portable certificate checks and recreates the source metrics and normalized gzip files. Run `pdflatex-interaction=non`

`stopmode-halt-on-errorreport.tex` twice to rebuild the PDF. The delivered `report.tex` is standalone; it embeds the checker and the attributed arithmetic appendix.

The checker is a terminating recursive algorithm on finite certificates and terms, abstracting from finite machine resources. Python recursion or memory limits can reject very large valid certificates, as for the matched controls. Only zero iteration is normalized, so no full primitive-recursive program evaluator is trusted. The new implementation has not been proved extensionally correct by a proof assistant, and no global optimality claim is made.

Attribution and inherited foundations

The following five mathematical components are reproduced from the repository's delivered reference development, preserved in the archive under `reference/` with its MIT No Attribution license and provenance. They establish the inherited choice basis, ordinary bridge, scalar predecessor, equality reflection, and primitive-recursion compiler. Their raw syntax and larger rule inventory belong to that reference development; Sections 2–5 above prove the comparison with the present normal five-pattern kernel. They are supplied to make the arithmetic part of the interpretation inspectable without network access, not as new contributions of this report.

A Complementary introductions with one local assumption

The old paired introduction retains both the zero instance and an induction instance when its matrix is an equation. Both together are unnecessary. A certificate may choose either one, provided the induction choice is restricted to a literal equation. This changes neither the formula language nor the meaning of a formula. The omitted combination is recovered by a short nested introduction–elimination construction.

A.1 Complete native syntax and rule inventory

The scalar numerical language is

$$t ::= x \mid 0 \mid St \mid I(n, a; u.h).$$

Only h binds the state variable u; n and a do not. The update may contain fixed numerical parameters. The computation axioms in every retained context are

$$I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u]. \tag{I0, IS}$$

The argument also applies unchanged to the full fixed-description PR language with its projection, composition, and recursion equations. No numerical variable ranges over function descriptions, and no uniform evaluator is postulated.

Native formulas and reusable context clauses are

$$A ::= (s = t) \mid \mathsf{D}_{x,y}A, \qquad \langle A; y.B(y)\rangle, \quad y \notin \mathrm{FV}(A).$$

Bound variables are renamed before capture-free substitution. The two variables x, y in a paired formula are distinct. An ordinary assumption F is the constant clause $\langle H; v.F\rangle$, where v is absent from F and H is the fixed computation theorem $I(0, 0; u.0) = 0$. Thus ordinary hypothesis use is a clause Use preceded by this computation proof.

There are computation axioms and exactly the following inference patterns:

$$\frac{\Gamma \vdash A}{\Gamma \vdash B(t)}\ \mathsf{Use} \quad (\langle A; y.B(y)\rangle \in \Gamma),$$

$$\frac{\Gamma \vdash \mathsf{D}_{x,y}B(x, y) \qquad \Gamma, \langle B(a, 0); y.B(a, y)\rangle \vdash F}{\Gamma \vdash F}\ \mathsf{DE}, \qquad \frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)}\ \mathsf{Tr},$$

$$\frac{\Gamma, B(t, \eta_\varepsilon(z)) \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y}B(x, y)}\ \mathsf{DI}_\varepsilon, \qquad \eta_0(z) = 0, \quad \eta_1(z) = z. \tag{Choice DI}$$

Here ε is a certificate choice in $\{0,1\}$, and $\varepsilon = 1$ is permitted only when B has a literal equation root. An equation beneath a binder is not a literal equation. Call $\varepsilon = 0$ the *case choice* and $\varepsilon = 1$ the *progress choice*. Both retain exactly one ordinary formula assumption. Compound matrices have only the case choice.

In DI the eigenparameter z is fresh for Γ, the witness t, and every fixed parameter of B. In DE the eigenparameter a is fresh for Γ, the conclusion F, and every fixed parameter of B. In transport, $F(v)$ is any fixed native formula template. Displayed local assumptions and clauses are discharged. There are no primitive reflexivity, coverage, separation, ordinary logical, or additional induction rules.

We call this calculus $\mathcal{C}$. The original comparison calculus $\mathcal{N}$ has identical syntax, computation, Use, DE, and transport, but its introduction is

$$\frac{\Gamma, B(t,0), [B(t,z)]_{\mathrm{at}} \vdash B(t,Sz)}{\Gamma \vdash \mathsf{D}_{x,y} B(x,y)}. \tag{Old DI}$$

The bracket supplies $B(t,z)$ exactly at a literal equation.

A.2 Exact equivalence and the unused-witness construction

Lemma A.1 (Structural admissibility)**.** *Weakening, exchange, contraction, renaming, ordinary-assumption cut, and capture-free numerical substitution are admissible in $\mathcal{C}$.*

Proof. Induct on derivations. Rename local eigenparameters before inserting proofs or terms. Numerical substitution preserves a formula's root constructor, so an allowed progress choice remains allowed. The selected choice does not change under substitution or cut. For ordinary-assumption cut, replace each use of its constant clause by the supplied proof. Computation schemas are closed under capture-free numerical substitution. These facts verify every case. □

Theorem A.2 (Identity-language equivalence)**.** *For every retained clause context Γ and native formula F,*

$$\Gamma \vdash_{\mathcal{C}} F \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{N}} F.$$

The transformations are effective on finite derivations and leave all numerical and formula syntax unchanged.

Proof. Each new introduction is an instance of the old one after weakening by the omitted local assumption. For the case choice at an equation, add the induction instance. For the progress choice, add the zero instance. Compound case introduction is already the old rule. Thus every new derivation translates to an old one.

For the reverse direction, transform an old derivation recursively. Only its atomic DI case needs a construction. Suppose its matrix is the equation $E(x,y)$ and its transformed premise is

$$\pi: \quad \Gamma, E(t,0), E(t,z) \vdash E(t,Sz).$$

The outer new inference will be case DI with witness t. It is therefore enough to prove $E(t,Sz)$ under $\Gamma, E(t,0)$.

Rename the eigenvariable in π to a fresh v. Choose a new variable u absent from $E(t,y)$ and all retained data. Under $\Gamma, E(t,0)$, progress DI with witness zero introduces

$$\mathsf{D}_{u,y} E(t,y)$$

from exactly the renamed premise $\pi[v/z]$. The bound coordinate u is deliberately unused.

Apply DE to this paired formula. Because u is unused, the opened clause is exactly

$$\langle E(t,0); y.E(t,y)\rangle;$$

it contains no new existential witness. The retained ordinary assumption $E(t,0)$ supplies its antecedent. Use at Sz gives $E(t,Sz)$, and DE closes. Finally the outer case DI introduces the original $\mathsf{D}_{x,y}E(x,y)$ with witness t. The original premise proof π has been inserted once.

An old compound introduction already is the new case choice. All other rule cases are unchanged. This completes the induction on the original derivation. □

The unused inner coordinate is essential. Introducing $\mathsf{D}_{x,y}E(x,y)$ internally and then eliminating it would give an arbitrary fresh witness, not necessarily the original t. The construction instead keeps t as a fixed parameter of $E(t,y)$ and binds only the unused coordinate u.

Proposition A.3 (Linear bound on inference nodes)**.** *Count each computation axiom, Use, DI, DE, and transport as one inference node, expanding ordinary-assumption use as a computation of H followed by Use. If an old derivation has N nodes and r atomic DI nodes, the displayed transformation has exactly*

$$N + 5r \leq 6N$$

nodes.

Proof. Each atomic old DI node is replaced by six nodes: outer case DI, inner progress DI, DE, Use of the opened clause, Use of the retained base assumption, and the computation of H needed for that last ordinary-assumption use. Its old premise subtree occurs once. All other inference nodes are copied once. □

This counts inference-tree nodes. It is not a bound on serialized JSON bytes, formula sizes, checker runtime, or the number of consumed packet cells when hypothesis closure is implicit. In the formula-context Q/D presentation, the corresponding replacement has outer case DI, inner progress DI, DE, QE, and two Hyp nodes, giving the same six-node count.

A.3 Induction and numerical cases separate cleanly

Lemma A.4 (Contextual equation induction)**.** *The progress choice, DE, and Use derive*

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)} \tag{EI}$$

for every equation E, with the usual freshness condition.

Proof. Choose an unused witness coordinate u. Progress DI introduces $\mathsf{D}_{u,y}E(y)$ from the given step. DE opens the clause $\langle E(0); y.E(y)\rangle$. Supply its antecedent using the given base proof and use the clause at t. Close DE. □

Lemma A.5 (Cases, equality, and binder congruence)**.** *The case choice derives, for arbitrary native formulas,*

$$\frac{\Gamma \vdash F(0) \qquad \Gamma \vdash F(Sz)}{\Gamma \vdash F(t)} \tag{NC}$$

with the usual freshness condition. Equality reflexivity, symmetry, transitivity, numerical congruence, and uniform matrix interderivability under D are derivable without using the progress choice.

Proof. For cases, use case DI with an unused witness coordinate and matrix $F(y)$, ignoring its retained base in the successor premise. DE opens $\langle F(0); y.F(y)\rangle$; the given zero proof and Use at t finish.

For reflexivity put $r = I(0, t; u.0)$. Use the computation equation $r = t$ as both transport premises with motive $v = t$ to obtain $t = t$. The usual explicit equality motives give symmetry, transitivity, and numerical congruence.

For binder congruence, open the old paired formula at a fresh witness a and introduce the new one by case DI at a. Convert its retained base to the old base, use the opened old clause at the successor, and convert back. Ordinary cut closes the construction, and the reverse conversion gives the converse. No induction is used. □

The primitive progress choice is still restricted to equations. Allowing it at arbitrary compound matrices would, by the argument for (EI), supply induction for arbitrary formulas and change the intended theory.

A.4 The entire ordinary logical interface

Here is a predecessor-free reconstruction of the ordinary logical operations. It makes explicit that the case choice supplies the logical interface independently of progress induction.

Let $C(v, a, b) := I(v, a; u.b)$ with u unused. Computation gives $C(0, a, b) = a$ and $C(Sv, a, b) = b$. Define the gate on fixed finite formula syntax by

$$G_v(s = t) := (C(v, 0, s) = C(v, 0, t)), \qquad G_v(\mathsf{D}_{x,y}B) := \mathsf{D}_{x,y}G_vB,$$

renaming bound variables away from the control. Equality, binder congruence, and external structural induction give

$$\vdash G_0A, \qquad G_{Sv}A \dashv\vdash A, \qquad A \dashv\vdash \mathsf{D}_{u,y}G_yA \quad (u, y \notin \mathrm{FV}(A)). \tag{Gate and padding}$$

For padding, case DI proves the forward direction. DE, the zero-gate theorem, and Use at 1 prove the reverse direction.

Define a selector between two arbitrary formulas by

$$M_v(s = t, u = w) := (C(v, s, u) = C(v, t, w))$$

and, with aligned fresh paired variables, by

$$\begin{aligned} M_v(\mathsf{D}_{x,y}A, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(A, B),\\ M_v(E, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(G_yE, B),\\ M_v(\mathsf{D}_{x,y}A, E) &:= \mathsf{D}_{x,y}M_v(A, G_yE), \end{aligned}$$

where E is an equation. Every call decreases maximum input binder depth. Padding and congruence prove $M_0(A, B) \dashv\vdash A$ and $M_{Sv}(A, B) \dashv\vdash B$.

Put $\delta(v) = C(v, 1, 0)$ and $p(z) = I(z, 0; u.\delta(u))$. Only the fixed computations $p(1) = 1$ and $p(2) = 0$ are needed. The full logical macro inventory is

$$\begin{aligned} A \Rightarrow_* B &:= \mathsf{D}_{u,z}M_z(A, B),\\ A \wedge_* B &:= \mathsf{D}_{u,z}G_zM_{p(z)}(A, B),\\ \exists_* x\, A(x) &:= \mathsf{D}_{x,y}G_yA(x),\\ A \vee_* B &:= \exists_* x\, M_x(A, B),\\ \forall_* x\, A(x) &:= A(0) \wedge_* \mathsf{D}_{u,z}G_zA(z),\\ \bot_* &:= (0 = 1). \end{aligned} \tag{Logical macros}$$

Newly bound variables are fresh; u is unused.

Implication introduction obtains its assumption from the zero selector and supplies the consequent at successors, then uses case DI. Elimination opens its clause and uses it at 1. Existential introduction uses its witness and the successor gate; elimination opens at a fresh witness, supplies the zero-gate theorem, and uses the clause at 1.

Disjunction introduction uses witnesses 0 and 1. For elimination, discharge the two branch proofs as $A \Rightarrow_* F$ and $B \Rightarrow_* F$. NC applied to $M_z(A, B) \Rightarrow_* F$ gives this implication at every control: the two endpoint cases follow from the branch implications. Existential elimination provides $M_a(A, B)$ and the implication gives F.

Given A and B, selector endpoints and NC give $M_v(A, B)$ at an arbitrary control. Substitute $v = p(Sz)$ and use the successor gate for conjunction introduction. For projections, open its zero-gated clause and specialize at 1 and 2; the two fixed computations give B and A, respectively.

These rules prove existential numerical coverage:

$$(x = 0) \vee_* \exists_* v\, (x = Sv). \tag{Coverage}$$

Use NC. Zero takes the left injection of reflexivity, and a successor takes the right injection with the evident witness. Coverage gives equality cases even when the retained context contains x; its elimination adds $x = 0$ or $x = Sv$ rather than silently changing that retained context.

Universal introduction follows from a fresh-parameter proof by substituting zero and successors and forming the two conjuncts. For elimination, take its $A(0)$ conjunct and open the paired conjunct. Its base is a zero-gate theorem. Coverage of the target chooses the zero instance from $A(0)$ or a successor instance from the opened clause and gate removal.

Finally, under $0 = 1$, computation and transport give $s = C(0, s, t) = C(1, s, t) = t$ for arbitrary numerical terms. Every equation follows, and external induction on formula syntax with case DI gives every native formula. This supplies falsity elimination. No excluded middle for quantified formulas has entered the construction.

A.5 Whole-domain realizations and both formula composites

For a standard first-order reading, let T_{eq} have intuitionistic equality logic, the same numerical signature and computation equations, numerical coverage, separation $(0 = 1) \to \bot$, and contextual equation induction. Its interpretation of the native syntax is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y} B)^\circ = \exists x \big(B^\circ(x, 0) \to \forall y B^\circ(x, y)\big),$$

$$\langle A; y.B \rangle^\circ = A^\circ \to \forall y B^\circ.$$

The domain is the entire numerical domain and interpreted equality is actual equality.

Case DI is sound in this realization by numerical coverage. Progress DI is sound by contextual equation induction under the conditional base. Use, DE, transport, and computation have their displayed ordinary meanings. Conversely, the native macros derive every ordinary logical rule, (EI) derives equation induction, and (Coverage) derives coverage. Separation translates to the identity implication $(0 = 1) \Rightarrow_* (0 = 1)$.

For the essential paired formula composite,

$$\mathsf{D}_{x,y} B \dashv\vdash \exists_* x \big(B(x, 0) \Rightarrow_* \forall_* y\, B(x, y)\big). \tag{Paired composite}$$

Forwards, DE opens its clause at a fresh a; under $B(a, 0)$ that clause supplies $B(a, y)$ at a fresh arbitrary y. Introduce the universal, implication, and existential macros and close DE. Backwards,

eliminate the existential at a fresh a. Introduce the paired formula with witness a by case DI: the retained base, supplied implication, and universal elimination at Sz give its successor premise.

The ordinary interpretations of the six logical macros have their ordinary round trips. For conjunction, its true base and the instances $1, 2$ recover both operands; the converse uses selector cases. The universal has all successor instances in its paired conjunct and zero in its first conjunct; coverage gives exactly the ordinary universal. Implication, disjunction, and existential quantification use their selector or gate endpoints. Falsity uses the explicit ordinary separation bridge. External induction on finite formulas, together with (Paired composite), gives both formula composites. Clauses correspond in both directions to $A \Rightarrow_* \forall_* yB$ by the derived logical rules.

The numerical composites are literal identities with internally provable graph $x = y$. No standard-numeral restriction, retract, quotient, or equality reflection assumption is being inserted at this stage. The separately supplied arithmetic bridge identifies T_{eq} with full iPRA; the present change preserves that bridge unchanged because Theorem A.2 preserves every native sequent.

A.6 What the theorem does and does not claim

The new inference interface retains one ordinary assumption at every paired introduction. It preserves the entire consequence relation, not just equations or closed theorems. Consequently the established full-iPRA interpretations and equiconsistency transfer unchanged.

This result is different from retaining only progress introduction and strengthening closure elimination. Here the original elimination rule is retained, and the case and progress introductions coexist. It is also different from allowing arbitrary compound progress, which would supply stronger induction.

A complete-checker comparison must still count the certificate choice, its equation-root restriction, substitution, context handling, computation, and the public input/output interface. The mathematical theorem alone is not a byte-count or minimality theorem. The inference-node bound is distinct from source-size measurements and from serialized certificate size. The proof is a written universal proof; finite regression tests do not constitute a Lean/Rocq formalization of it.

B Arithmetic strength and both ordinary interpretation composites

We spell out the arithmetic bridge to avoid identifying standard numerical agreement with bi-interpretability. First elaborate every fixed iterator into an ordinary function symbol. At stage zero the symbols are $0, S$. Given a fixed earlier-stage term $h(u, \vec{p})$, introduce $i_h(n, a, \vec{p})$ with the two equations

$$i_h(0, a, \vec{p}) = a, \qquad i_h(Sn, a, \vec{p}) = h(i_h(n, a, \vec{p}), \vec{p}).$$

Only finite descriptions occur; the domain contains no code for a universal evaluator. Formal parameters are distinct when a description is declared, while actual arguments can coincide.

Let T_I be ordinary intuitionistic first-order logic with actual equality and genuine falsity, these staged equations, contextual induction for literal equations, and

$$\forall x\,(x = 0 \vee \exists y\, x = Sy), \qquad (0 = S0) \to \bot.$$

The second sentence is the falsity bridge. It must be stated: positive numerical equations by themselves admit a singleton structure. Native explosion from $0 = S0$ and a translation of ordinary falsity to that equation do not authorize silently dropping the bridge from T_I.

The native-to-ordinary translation is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y}B)^\circ = \exists x\big(B^\circ(x, 0) \to \forall y B^\circ(x, y)\big).$$

A clause $\langle A; y.B\rangle$ becomes $A^\circ \to \forall y B^\circ$. For the auxiliary closure used by the compact checkers, $(\mathsf{Q}_y B)^\circ = B^\circ(0) \to \forall y B^\circ$. The interpretation domain is all elements, defined by $x = x$; equality is actual equality.

Case DI is sound because numerical coverage combines its retained base with its successor proof. Progress DI is sound by literal-equation induction under a temporarily assumed base. Use, DE and transport are ordinary logical inferences under the translations. Conversely the derived logical operations, equation induction, cases and explosion in Section A recover the rules and axioms of T_I. An auxiliary closure is interderivable with $\mathsf{D}_{v,y}B(y)$ for unused v: in one direction use case DI with the closure and its base; in the other eliminate the paired formula and return its supplied closure. Thus formulas and context clauses have the stated meanings in both presentations.

B.1 The arithmetic dependency order

The scalar update has no recursion-counter argument. We do not conceal one in an encoding. The inherited internal proofs, reproduced in the appendices, establish the required arithmetic in this order:

1. Using equation induction and numerical cases, construct a fixed scalar predecessor term and prove $P0 = 0$ and $P(Sn) = n$. Existential coverage then gives $x = 0 \vee x = S(Px)$: in the successor case $x = Sy$, apply P to obtain $Px = y$ and substitute.
2. Construct truncated subtraction and a numerical characteristic of actual equality. Prove its reflection on the entire original domain.
3. Compile each fixed quantifier-free formula into its characteristic equation and recover every contextual quantifier-free induction instance.
4. With that induction available, construct pairing and projections by a bounded square-root search and compile every fixed general primitive-recursive description into a scalar iterator term.

No pairing or faithful equality detector is assumed to obtain predecessor. No appeal to standard-model correctness replaces an internal equality. Every induction fixes its passive parameters; diagonal substitution occurs only into a previously proved identity.

For a target recursion

$$f(0, \vec{p}) = g(\vec{p}), \qquad f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p}),$$

let π, L, H be the proved pairing and projections. The compiler uses

$$V(n, \vec{p}) = I\big(n, \pi(0, g^*(\vec{p})); u.\pi(SL(u), h^*(L(u), H(u), \vec{p}))\big), \qquad f^*(n, \vec{p}) = H(V(n, \vec{p})).$$

Equation induction proves $L(V(n, \vec{p})) = n$; the two projection identities then prove the target defining equations. The construction is external recursion on each finite PR description, not an internal quantification over arbitrary functions.

Theorem B.1 (Whole-domain bi-interpretability and equiconsistency)**.** *The ordinary realization T_I and full first-order iPRA are bi-interpretable. In both theories $x = y$ defines the isomorphism from the original domain to its double interpretation. The native Choice-DI calculus is therefore a faithful proof presentation of a theory bi-interpretable and equiconsistent with iPRA.*

Proof. Interpret an iterator in iPRA by primitive recursion that ignores its counter argument in the update. iPRA proves its equations, literal equation induction, coverage and separation. In the reverse direction use the scalar compiler. The predecessor and equality-reflection proofs supply the successor axioms and every quantifier-free induction instance. All ordinary logical rules are preserved.

For two iterators with the same seed and provably equal updates, equation induction on the equality of their values proves uniqueness at every counter, with all parameters fixed. The corresponding argument applies to ordinary primitive recursion. Induct externally on a finite description: its double translation satisfies its original defining equations, so internal uniqueness identifies the two functions. This also proves coherence when substitution changes the staged syntactic description of an update.

The domain and equality are unchanged; hence the identity graph is internally total, bijective and preserves every interpreted operation. The displayed ordinary meanings of the logical macros and the paired binder give both formula composites by external induction on finite formula syntax. Clauses are similarly equivalent to their conditional universal meanings. The explicit falsity bridge preserves contradiction in both directions. Effective translations of finite proofs give equiconsistency, without a claim that either theory proves its own consistency. □

C Internal scalar predecessor bootstrap

This scalar bootstrap is inherited from the supplied scalar development [8] and is reproduced to keep the scalar checker self-contained. It is not a new contribution of this paper. The proof uses a factor map fixing odd states and a saturated-iteration construction of predecessor.

The available arithmetic terms are 0, S, and

$$I(n, a; u.h), \qquad I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u].$$

The update binds only u. All other variables in it are fixed numerical parameters. We use equality reasoning, the already reconstructed zero/successor cases, and induction on a single literal equation. Every induction below keeps its other parameters fixed. Predecessor, successor injectivity, pairing, and induction on arbitrary quantifier-free formulas are not assumptions of the argument.

C.1 Elementary terms and two finite cycles

Write $1 = S0$ and use larger numerals as successor abbreviations. Define

$$C(n, a, b) = I(n, a; u.b), \qquad Z(n) = C(n, 1, 0), \qquad a + b = I(b, a; u.Su).$$

The update variable of C is absent from its body. Consequently

$$C(0, a, b) = a, \qquad C(Sn, a, b) = b.$$

Numerical cases show that $Z(n)$ is 0 or 1. The usual commutative-monoid equations for addition follow by literal equation induction. For example, $0 + b = b$ and $Sa + b = S(a + b)$ are proved by induction on b; these yield commutativity by a further equation induction, and associativity follows by induction on the last summand. We use these equations to reorder finite sums. The notation $2a$ means $a + a$ and $3a$ means $a + a + a$; no multiplication operation is needed for this bootstrap.

Put

$$N(n) = I(n, 0; u.Z(u)), \qquad j(u) = Z(u) + 2N(u), \qquad M(n) = I(n, 0; u.j(u)).$$

The range of N is contained in $\{0,1\}$: its zero value is 0, and its successor value is a value of Z. This uses numerical cases on its counter, not induction on a disjunction. Hence

$$N(n) + Z(N(n)) = 1, \qquad Z(Z(N(n))) = N(n), \qquad N(n+2) = N(n). \tag{1}$$

Here the last equation follows from two successor computations and the preceding equation. Literal-equation inductions give

$$N(2n) = 0, \qquad N(2n+1) = 1, \qquad N(3n) = N(n). \tag{2}$$

For the first equation the successor step adds two to the argument of N; the second follows by one computation. In the third, increasing n adds three to the left argument and therefore applies Z once to both sides of the induction equation.

Finite computations now give

$$j(0) = 1, \qquad j(1) = 2, \qquad j(2) = 0.$$

The proof below needs just these three values and $M(Sn) = j(M(n))$; it does not assume a division algorithm or a theorem characterizing M as remainder modulo three.

C.2 A ramp with a global scaling identity

Define

$$G(n) = I(n, 0; u.u + M(u) + 1).$$

The central invariant is

$$M(G(n)) = N(n). \tag{3}$$

Its zero case is immediate. For its successor step, retain the single induction equation and split the Boolean value $N(n)$. If it is 0, the next G state is $G(n)+1$; its M value is $j(0) = 1$, equal to $N(Sn)$. If it is 1, the next G state is $G(n) + 2$; two computations of M change its value from 1 to 2 and then to 0, again equal to $N(Sn)$. This proves the invariant by literal-equation induction.

It follows that

$$G(Sn) = G(n) + N(n) + 1, \qquad G(n+2) = G(n) + 3.$$

Equation induction therefore yields

$$G(2n) = 3n. \tag{4}$$

A separate literal-equation induction proves

$$G(3n) = 3G(n) + N(n). \tag{5}$$

For completeness, its successor calculation is

$$\begin{aligned} G(3Sn) &= G(3n) + N(3n) + 4 = 3G(n) + 2N(n) + 4, \\ 3G(Sn) + N(Sn) &= 3G(n) + 3N(n) + 3 + Z(N(n)). \end{aligned}$$

The expressions agree in both Boolean cases for $N(n)$. The base is $0 = 0$. In particular, (5) was proved internally; it was not inferred from an intended floor-function interpretation of G.

Set

$$H(u) = C(N(u), G(u), u).$$

The crucial choice is to retain u when its parity is 1. The preceding equations give

$$H(2u) = 3u, \qquad H(1) = 1, \qquad H(3u) = 3H(u). \tag{6}$$

To check the last identity, use $N(3u) = N(u)$ and split the Boolean $N(u)$. At 0, both sides are $3G(u)$ by (5); at 1, both are $3u$. This argument does not require that an arbitrary number has already been represented as $2v$ or $2v+1$.

C.3 A comparator from scalar iteration

For every fixed update h, equation induction on n, keeping m, a and all parameters fixed, proves the semigroup identity

$$I(m+n,a;h) = I(n, I(m,a;h);h).$$

The base is the zero computation and the step applies h to the induction equation. This identity will also be used in the later pairing construction.

For a fixed update h, equation induction on y, with its initializer a and all parameters of h fixed, proves the rotation identity

$$I(Sy,a;h) = I(y,h(a);h). \tag{7}$$

Both sides have base $h(a)$, and their successor equations apply the same h to the previous equality. Here and below $I(y,a;h)$ is an abbreviation for one fixed finite scalar term, not quantification over functions.

By (6), another equation induction gives

$$I(y,3a;H) = 3I(y,a;H). \tag{8}$$

Its base is $3a = 3a$. In the step the fixed-parameter induction equation rewrites the new left side as

$$H(3I(y,a;H)) = 3H(I(y,a;H)).$$

The initializer parameter stays a throughout this induction. Equation induction also gives $I(y,1;H) = 1$, since $H(1) = 1$.

Define

$$E(x) = I(x,1;u.2u), \qquad L(x,y) = N\big(I(y,E(x);H)\big).$$

Then the following entire recursion specification is provable:

$$L(0,y) = 1, \qquad L(Sx,0) = 0, \qquad L(Sx,Sy) = L(x,y). \tag{9}$$

The first equation uses $E(0) = 1$ and the fixed-point identity for H. The second uses $E(Sx) = 2E(x)$ and $N(2a) = 0$. For the third,

$$\begin{aligned} L(Sx,Sy) &= N\big(I(Sy,2E(x);H)\big) \\ &= N\big(I(y,H(2E(x));H)\big) \\ &= N\big(I(y,3E(x);H)\big) \\ &= N\big(3I(y,E(x);H)\big) \\ &= N\big(I(y,E(x);H)\big) = L(x,y). \end{aligned}$$

The successive steps use rotation, $H(2a) = 3a$, iterator scaling, and $N(3a) = N(a)$. Thus L has the comparison equations at all arguments internally. Its standard-model meaning as the indicator of $x \le y$ is motivation, not a premise of the following argument.

C.4 Saturated iteration gives predecessor

We now need only the three equations (9). Define

$$B_m(u) = C(L(Su,m),u,Su), \qquad W(t,m) = I(t,0;u.B_m(u)).$$

The subscript in B_m records a fixed parameter; B_m is an open term abbreviation, not a function variable.

Numerical cases on e prove the selector equation

$$C(e, Sa, Sb) = SC(e, a, b).$$

Equivalently one may use equation induction on e, whose induction hypothesis is unused. Therefore

$$\begin{aligned} B_{Sm}(Su) &= C(L(SSu, Sm), Su, SSu) \\ &= C(L(Su, m), Su, SSu) = SB_m(u), \qquad (10) \\ B_{Sm}(0) &= C(L(S0, Sm), 0, S0) = C(L(0, m), 0, S0) = S0. \qquad (11) \end{aligned}$$

Literal-equation induction on t, with m fixed, proves

$$W(St, Sm) = SW(t, m). \qquad (12)$$

The base is (11). The successor step is

$$\begin{aligned} W(SSt, Sm) &= B_{Sm}(W(St, Sm)) \\ &= B_{Sm}(SW(t, m)) \\ &= SB_m(W(t, m)) = SW(St, m). \end{aligned}$$

Only the second line uses the current induction hypothesis, at the same fixed m.

Two subsequent equation inductions give

$$L(Sn, n) = 0, \qquad W(n, n) = n. \qquad (13)$$

The first uses the comparator recurrence. The second uses (12) in its successor step. The substitution $t = m = n$ is made into that already proved universally parameterized equation; no parameter of an active induction hypothesis is changed. Consequently

$$W(Sn, n) = B_n(n) = C(L(Sn, n), n, Sn) = n. \qquad (14)$$

Finally define the finite scalar term

$$P(m) = I\big(m, 0; u.C(L(SSu, m), u, Su)\big). \qquad (15)$$

Its zero equation is immediate. To justify its successor equation without assuming a separate extensionality rule for iterator bodies, put

$$V(t, m) = I\big(t, 0; u.C(L(SSu, Sm), u, Su)\big).$$

The comparator recurrence identifies this update pointwise with $B_m(u)$. Equation induction on t, with m fixed, gives $V(t, m) = W(t, m)$: their base values are 0, and the successor step uses the pointwise update equation and then the induction equation. Now (14) yields

$$\boxed{P(0) = 0, \qquad P(Sm) = V(Sm, m) = W(Sm, m) = m.}$$

C.5 Dependency boundary

All terms in this section are built from $0, S, I$ by finite substitution. All inductions have a single equation as their invariant. Numerical cases and equality transport are the already reconstructed logical resources. Successor injectivity may now be obtained by applying P to an assumed equation $Sx = Sy$. Equality tests, reflection of quantifier-free formulas, pairing, and the recovery of general primitive recursion can then be developed in the established order. They were not used in order to manufacture predecessor.

This argument is a written internal derivation in the stated theory. The finite implementation checks do not constitute a proof-assistant formalization of this entire metatheorem.

D From literal-equation induction to full intuitionistic PRA

This section supplies an unconditional completion theorem. It is adapted with attribution from the current forward-left draft [9]. In the scalar application used here, every function occurring in the proof is already a scalar term: predecessor is supplied by Appendix C, the selector is $C(n, a, b) = I(n, a; u.b)$, addition is $x + y = I(y, x; u.Su)$, and truncated subtraction is $d(x, y) = I(y, x; u.Pu)$. All other functions below are finite compositions of these. Thus this proof does not presuppose that arbitrary primitive recursion has already been compiled. The QFI conclusion first applies to the scalar language; the general PR compiler comes afterward. Literal-equation induction, together with the primitive-recursive defining equations, predecessor coverage, and separation of 0 and 1, proves induction for every quantifier-free formula. In particular, the proof does not assume a faithful numerical equality test, stability or decidability of equality, successor uniqueness, or any commutation law for an iterator. We construct the equality test and prove its reflection property for arbitrary elements of the numerical domain.

D.1 The exact hypotheses

Work in ordinary intuitionistic first-order logic with equality and genuine falsity $\bot$. The language has one numerical sort and a symbol for each fixed, finite primitive-recursive description. Distinct descriptions are not identified merely because they compute the same standard function. Let PRE contain the defining equations for zero, successor, projections, composition, and primitive recursion. In particular, choose descriptions P and C with equations

$$P(0) = 0, \qquad P(Sx) = x, \tag{16}$$

$$C(0, a, b) = a, \qquad C(Sx, a, b) = b. \tag{17}$$

Write $1 = S0$. In addition to PRE, assume

$$\mathsf{Cov}: \quad \forall x \, (x = 0 \vee x = S(Px)), \tag{18}$$

$$\mathsf{Sep}: \quad 0 = 1 \longrightarrow \bot, \tag{19}$$

and induction for each *literal equation* $E(n, \vec{p})$:

$$\forall \vec{p} \, \left[(E(0, \vec{p}) \wedge \forall n (E(n, \vec{p}) \longrightarrow E(Sn, \vec{p}))) \longrightarrow \forall n \, E(n, \vec{p}) \right]. \tag{20}$$

Here the schematic expression E is a single equality of numerical terms; the displayed universal quantifiers range only over numbers. Every description and every instance of the schema is finite external syntax.

The axiom formulation (20) and the usual contextual induction rule are interchangeable. For example, if

$$\Gamma \vdash E(0,\vec{p}), \qquad \Gamma, E(z,\vec{p}) \vdash E(Sz,\vec{p}),$$

where z is absent from Γ and the passive parameters, discharge the second premise's equation and universally introduce z. The axiom then gives $\Gamma \vdash \forall n E(n,\vec{p})$, which can be instantiated at any target term. Conversely, assume the antecedent of the axiom and apply the contextual rule with fresh variables. This uses the ordinary intuitionistic logical rules and permits arbitrary, including quantified, formulas in Γ.

D.2 Constructor separation and two elementary tests

Define the sign description and the countdown description by

$$\mathrm{sg}(u) = C(u,0,1), \qquad K(u) = C(u,1,Pu). \tag{21}$$

Their defining equations imply

$$\mathrm{sg}(0) = 0, \qquad \mathrm{sg}(Sx) = 1, \tag{22}$$
$$K(0) = 1, \qquad K(Sx) = x. \tag{23}$$

Constructor laws. Successor is injective: applying P to $Sx = Sy$ gives $x = y$ by (16). Successor never equals zero. Indeed, $Sx = 0$ gives $1 = 0$ by applying sg and using (22); symmetry and (19) give $\bot$. Consequently $1 \neq 0$, where $A \neq B$ abbreviates $A = B \to \bot$. None of these arguments uses induction.

The sign test. For every u,

$$\mathrm{sg}(u) = 0 \vee \mathrm{sg}(u) = 1, \tag{24}$$
$$\mathrm{sg}(u) = 0 \quad \longleftrightarrow \quad u = 0, \tag{25}$$
$$\mathrm{sg}(u) = 1 \quad \longleftrightarrow \quad u \neq 0. \tag{26}$$

To prove the range assertion, split (18). The branch $u = 0$ gives sign 0, and the branch $u = S(Pu)$ gives sign 1. The implication $u = 0 \to \mathrm{sg}(u) = 0$ is computation. For its converse, the successor branch of coverage would make $\mathrm{sg}(u) = 0$ imply $1 = 0$, a contradiction; in that branch explosion gives $u = 0$ as well. Similarly, if $\mathrm{sg}(u) = 1$ and $u = 0$, computation gives $0 = 1$. Conversely, $u \neq 0$ excludes the zero branch of coverage, and the successor branch computes the sign as 1.

Coverage and (17) also give, for all u, v,

$$C(u,v,v) = v. \tag{27}$$

Both coverage branches compute to v, so this identity again needs no induction.

The countdown test and the notation W. For every u,

$$K(u) = 0 \quad \longleftrightarrow \quad u = 1. \tag{28}$$

If $u = 1 = S0$, (23) gives $K(u) = 0$. Conversely, assume $K(u) = 0$ and split coverage. If $u = 0$, then $K(u) = 1$, contradicting separation, and explosion gives $u = 1$. If $u = S(Pu)$, computation gives $K(u) = Pu$; hence $Pu = 0$ and $u = S0 = 1$.

If a presentation uses a description named W with defining equations

$$W(0) = 1, \qquad W(Sx) = x,$$

then it proves $K(u) = W(u)$ for every u already from coverage. In the zero branch both sides are 1; in the branch $u = S(Pu)$ both sides are Pu. Thus K and W are provably equal on the whole domain, even if they are distinct descriptions. This equality does not appeal to standard-model agreement or to uniqueness of an arbitrary recurrence.

D.3 Four elementary uses of equation induction

Choose descriptions for addition and truncated subtraction, with recursion on the second argument:

$$x + 0 = x, \qquad x + Sy = S(x + y), \tag{29}$$
$$d(x, 0) = x, \qquad d(x, Sy) = P(d(x, y)). \tag{30}$$

Using these equations and (20), we now derive

$$0 + y = y, \tag{31}$$
$$(Sx) + y = S(x + y), \tag{32}$$
$$d(Sx, Sy) = d(x, y), \tag{33}$$
$$d(Sa, a) = 1. \tag{34}$$

Each induction below has a single equation as its induction formula.

For (31), induct on y. The base is $0 + 0 = 0$. The step is $0 + Sy = S(0 + y) = Sy$.

For (32), hold x fixed and induct on y. At zero both sides are Sx. At a successor the induction hypothesis gives

$$(Sx) + Sy = S\big((Sx) + y\big) = S\big(S(x + y)\big) = S(x + Sy).$$

Combining (32) with the defining equation for addition gives the useful rearrangement

$$(Sy) + r = y + Sr. \tag{35}$$

Indeed, both sides are $S(y + r)$.

For (33), hold x fixed and induct on y. The base computes as

$$d(Sx, S0) = P(d(Sx, 0)) = P(Sx) = x = d(x, 0).$$

Applying P to the induction equation gives the step:

$$d(Sx, S(Sy)) = P(d(Sx, Sy)) = P(d(x, y)) = d(x, Sy).$$

Finally, induct on a for (34). The base is $d(S0, 0) = S0 = 1$. At a successor,

$$d(S(Sa), Sa) = d(Sa, a) = 1,$$

using (33) and the induction hypothesis.

D.4 An equation invariant that reflects actual equality

The remaining arithmetic ingredient is the following equation, valid for all x, y:

$$C(\mathrm{sg}(d(x,y)),\ x,\ y + d(x,y)) = x. \tag{36}$$

Its meaning is that a nonzero subtraction residual reconstructs the original argument by addition. The selector expresses this assertion as one literal equation, so equation induction applies directly.

Hold x fixed and induct on y. At $y = 0$, subtraction gives residual x, and (31) gives $0 + x = x$. The desired equation is consequently $C(\mathrm{sg}(x), x, x) = x$, which is (27).

For the step, assume (36) at y and abbreviate

$$r = d(x,y), \qquad r' = Pr = d(x, Sy).$$

These are local term abbreviations, not additional function symbols. Split coverage for r'.

If $r' = 0$, then $\mathrm{sg}(r') = 0$, and the successor instance of (36) reduces immediately to $x = x$.

In the other branch, $r' = S(Pr')$, so constructor separation gives $r' \neq 0$. Split coverage for r. Its zero branch would imply $r' = P0 = 0$, a contradiction, from which the desired step equation follows. In its successor branch,

$$r = S(Pr) = Sr', \qquad \mathrm{sg}(r) = 1.$$

The induction equation therefore computes to $y + r = x$. Meanwhile, (35) gives

$$(Sy) + r' = y + Sr' = y + r = x.$$

Since r' is a successor, $\mathrm{sg}(r') = 1$, so this is exactly the branch selected in the successor instance of (36). This completes the equation induction. All case distinctions came from coverage; no induction on an implication or a disjunction was used.

Reflection at residual 1. We have, for arbitrary a, b,

$$d(Sa, b) = 1 \quad \longleftrightarrow \quad a = b. \tag{37}$$

For the forward direction, substitute $x = Sa$, $y = b$ into (36). If $d(Sa, b) = 1$, the sign is 1 and the selector returns its second branch, giving

$$b + 1 = Sa.$$

The defining equation for addition gives $b + 1 = S(b + 0) = Sb$. Thus $Sb = Sa$, and applying P gives $b = a$, hence $a = b$. Conversely, $a = b$ and (34) give $d(Sa, b) = d(Sa, a) = 1$.

Define the equality characteristic by the fixed description

$$e(a,b) = \mathrm{sg}\big(K(d(Sa,b))\big). \tag{38}$$

The sign, countdown, and residual reflection lemmas give

$$e(a,b) = 0 \quad \longleftrightarrow \quad a = b, \tag{39}$$

$$e(a,b) = 0 \ \vee\ e(a,b) = 1. \tag{40}$$

In detail, $e(a,b) = 0$ is equivalent to $K(d(Sa,b)) = 0$ by (25), which is equivalent to $d(Sa,b) = 1$ by (28), and then to $a = b$ by (37). The range assertion is (24) applied to $K(d(Sa,b))$.

The reflection property concerns the logic's actual equality throughout the domain. It was not assumed as part of a numerical comparison symbol, and its proof did not restrict a or b to standard numerals. It also yields decidable equality. In the $e(a,b) = 0$ branch, (39) gives $a = b$. In the $e(a,b) = 1$ branch, an assumption $a = b$ would give $e(a,b) = 0$ and therefore $0 = 1$, contradicting separation. Hence

$$\forall a\, \forall b\, \big(a = b \vee (a = b \to \bot)\big) \tag{41}$$

is a theorem, not an extra hypothesis.

D.5 Compilation of all quantifier-free formulas

For each fixed quantifier-free formula $A(\vec{p})$, construct a term $b_A(\vec{p})$ by external recursion on its finite syntax. Take equality, $\bot$, $\wedge$, $\vee$, and $\to$ as the primitive formula constructors; negation is implication into $\bot$. Set

$$b_{s=t} = e(s,t), \qquad b_\bot = 1, \tag{42}$$
$$b_{A\wedge D} = C(b_A, b_D, 1), \tag{43}$$
$$b_{A\vee D} = C(b_A, 0, b_D), \tag{44}$$
$$b_{A\to D} = C(b_A, b_D, 0). \tag{45}$$

Every term here is a finite composition of fixed primitive-recursive descriptions; if desired, projections and composition give it a single description symbol. No internal quantification over formulas or programs is involved.

An external induction on the syntax of A proves, within the arithmetic theory,

$$\big(b_A = 0 \vee b_A = 1\big) \quad \text{and} \quad \big(b_A = 0 \longleftrightarrow A\big). \tag{46}$$

Here are the full connective checks. In every case $b_A = 1$ implies $\neg A$: if A held, the already established implication $A \to b_A = 0$ would contradict separation.

Atomic formulas and falsity. For an equation, both claims are (39)–(40). For falsity the range assertion is $1 = 0 \vee 1 = 1$, with its right disjunct proved by reflexivity. Also $1 = 0 \to \bot$ follows from separation and symmetry, while $\bot \to 1 = 0$ follows by explosion. Thus $1 = 0$ is equivalent to $\bot$, as required.

Conjunction. Split $b_A = 0 \vee b_A = 1$. In the zero branch, A holds and $b_{A\wedge D} = b_D$. Its range follows from the range of b_D, and its being zero is equivalent to D, hence to $A \wedge D$ under the known A. In the one branch, $\neg A$ holds and $b_{A\wedge D} = 1$. Its range is immediate. If it were zero, separation would give $\bot$, hence $A \wedge D$; conversely an assumption $A \wedge D$ contradicts $\neg A$, and explosion gives $b_{A\wedge D} = 0$.

Disjunction. In the branch $b_A = 0$, the formula A holds and $b_{A\vee D} = 0$. The range is immediate, and both $b_{A\vee D} = 0$ and $A \vee D$ hold, proving their equivalence. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\vee D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D and hence $A \vee D$. Conversely, from $A \vee D$, the A alternative contradicts $\neg A$ and yields $b_D = 0$ by explosion, while the D alternative yields $b_D = 0$ directly. This proves the required equivalence in both alternatives.

Implication. In the branch $b_A = 0$, the formula A holds and $b_{A\to D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D, which gives $A \to D$. Conversely, $A \to D$ and the known A give D, hence $b_D = 0$. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\to D} = 0$. Its range is immediate. To prove $A \to D$, assume A; this contradicts $\neg A$, and explosion gives D. Thus both $b_{A\to D} = 0$ and $A \to D$ hold in this branch, and their equivalence follows. This argument uses the proved two-value range of the finite characteristic. It does not use excluded middle for arbitrary formulas or change the surrounding intuitionistic logic.

This completes the external syntactic induction proving (46) for every quantifier-free formula.

D.6 Recovery of induction and the full theory

Let $A(n, \vec{p})$ be any fixed quantifier-free formula. Suppose an arbitrary retained context Γ proves its base and its induction step:

$$\Gamma \vdash A(0, \vec{p}), \qquad \Gamma, A(z, \vec{p}) \vdash A(Sz, \vec{p}),$$

with the usual freshness condition on z. By (46), the base gives $\Gamma \vdash b_A(0, \vec{p}) = 0$. Under the additional assumption $b_A(z, \vec{p}) = 0$, the same equivalence gives $A(z, \vec{p})$, the given step gives $A(Sz, \vec{p})$, and the equivalence gives $b_A(Sz, \vec{p}) = 0$. Apply literal-equation induction to $b_A(n, \vec{p}) = 0$ and then apply its reflection implication. The result is $\Gamma \vdash A(t, \vec{p})$ for every target term t. Thus the theory proves the full contextual quantifier-free induction rule, including in arbitrary quantified contexts.

Completion theorem. In the common language of fixed primitive-recursive descriptions, the theory

$$\mathsf{PRE} + \mathsf{Cov} + \mathsf{Sep} + \mathsf{EI}$$

has exactly the same sequents as ordinary first-order intuitionistic primitive recursive arithmetic with all quantifier-free induction instances. The forward simulation follows from the recovered quantifier-free induction, successor injectivity, and successor separation; all primitive-recursive defining equations and all logical rules are shared. For the reverse simulation, literal-equation induction is an instance of quantifier-free induction, and successor separation gives Sep. Quantifier-free induction proves Cov: at zero choose its left disjunct, and at a successor use $Sx = S(P(Sx))$, which follows from $P(Sx) = x$, for its right disjunct.

Both interpretations therefore use the full domain, defined by $x = x$, and preserve every function symbol, equality, logical connective, and quantifier. Both composites have the identity isomorphism, with graph $x = y$. The theories are consequently identity-domain bi-interpretable and equiconsistent. This completion theorem is unconditional once the literal-equation induction rule has been obtained; it has no dependency on a classical successor-uniqueness reduction.

E Pairing, primitive recursion, and internal numerical composites

This appendix is adapted with attribution from the progress-closure arithmetic development [10]. Its prerequisites have now been proved: scalar predecessor, arithmetic reflection, and full quantifier-free induction. Pairing is a derived proof tool; it is not part of any checker.

E.1 Pairing from a bounded square-root search

Once QFI is available, a familiar square-based injection gives a particularly direct compiler. Write $a \le b$ for $d(a, b) = 0$. The usual linear-order and semiring identities follow by fixed quantifier-free

inductions from the successor, addition, and monus equations. Multiplication itself is the scalar term

$$a \cdot b = I(b, 0; u.u + a).$$

Only primitive-recursive arithmetic is involved in the following bounded root search; there is no minimization over an unbounded domain.

For a fixed parameter n, put

$$U_n(v) = C(d((Sv)^2, n), Sv, v), \qquad R_n(i) = I(i, 0; v.U_n(v)), \qquad q(n) = R_n(n).$$

The update increments v exactly when $(v+1)^2 \leq n$, and otherwise retains it. QFI in i, with n fixed, proves

$$R_n(i) \leq i, \qquad R_n(i)^2 \leq n, \qquad R_n(i) = i \ \vee \ n < (S(R_n(i)))^2. \qquad \text{(Root invariant)}$$

The base has $R_n(0) = 0$. If the update increments, its test contradicts the strict-upper-bound alternative in the third invariant. Thus the old state equals i, and the new state equals Si; its square bound is the increment test. If the update does not increment, the strict upper square bound holds and the state remains fixed. An already established strict upper square bound forces every subsequent update to remain fixed. These are exactly the cases needed for the three displayed invariants.

At $i = n$, either the strict upper square bound already holds or $R_n(n) = n$, in which case $n < (n+1)^2$ supplies it. Thus

$$q(n)^2 \leq n < (q(n)+1)^2. \qquad \text{(Root)}$$

Monotonicity of squares shows uniqueness of a number satisfying Root. All formulas just used are quantifier free in the scalar language, with fixed n during the induction.

Now define

$$\pi(a, b) = (a + b)^2 + a, \qquad L(n) = d(n, q(n)^2), \qquad H(n) = d(q(n), L(n)).$$

For $k = a + b$, one has $k^2 \leq k^2 + a < (k+1)^2$ since $a \leq k$. Root uniqueness gives $q(\pi(a, b)) = a + b$, and therefore

$$L(\pi(a, b)) = a, \qquad H(\pi(a, b)) = b.$$

Surjectivity of π is unnecessary. The original numerical domain is never replaced by the range of this coding; pairing is used only to store the finite state of a fixed primitive recursion. This standard square pairing is a convenient reconstruction here, not a claimed new arithmetic basis.

E.2 Compiling each primitive recursion

Suppose the fixed target description is

$$f(0, \vec{p}) = g(\vec{p}), \qquad f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p}).$$

Assume by external structural induction that the earlier g, h already have scalar translations g^*, h^*. Define

$$W(u, \vec{p}) = \pi(SL(u), h^*(L(u), H(u), \vec{p})),$$
$$V(n, \vec{p}) = I(n, \pi(0, g^*(\vec{p})); u.W(u, \vec{p})), \qquad f^*(n, \vec{p}) = H(V(n, \vec{p})).$$

The update has only its state variable bound, with every p_i fixed. Equation induction proves $L(V(n, \vec{p})) = n$. The two projection equations and scalar computation then prove both defining equations for f^* internally. Initial functions and composition translate directly. This completes the translation of every fixed finite PR description without adding a universal evaluator.

E.3 Why both numerical round trips are internal

Two fixed scalar iterators with the same seed and provably equal updates are internally equal at all counters: use literal-equation induction comparing their values. Their base agrees, and their successor equations plus congruence prove the step. The same uniqueness argument works for ordinary primitive recursion.

For an ordinary PR description, induct externally on its finite construction. The double translation satisfies its original defining equations, so internal equation induction identifies it with the original operation. For a native scalar description, earlier translated updates agree by the external induction hypothesis; scalar uniqueness identifies the double-translated iterator. These proofs hold at arbitrary internal counters.

Thus the domain relation $x = y$ is provably a total bijection preserving every fixed numerical operation in both composites. The formula constructions and round trips in Section A give the corresponding formula composites. Both interpretations preserve falsity, taking native $0 = 1$ to an ordinary contradiction and ordinary $\bot$ to native $0 = 1$. Finite proof translations therefore establish equiconsistency.

F The complete selected proof checker

The listing includes the whole standalone program; it imports no project code.

```
"""Standalone five-pattern clausal iPRA checker.

Terms are zero, bound positive indices, fresh negative levels, [0,t]
(successor), and [n,a,h] (iterator). Zero iterators are normalized away.
Formulas are [s,t] and [B] (paired binder). Contexts contain clauses.
A validated reflexive goal is a leaf regardless of its unused payload.
"""
import json
import sys


def require(condition):
    if not condition:
        raise ValueError('invalid certificate')


def term(a, scope, depth=0):
    if type(a) is int:
        return -scope <= a <= depth
    return (type(a) is list and len(a) in (2, 3)
            and (a[0] != 0 if len(a) == 3 else a[0] == 0)
            and all(term(t, scope, depth + (len(a) == 3 and i == 2))
                    for i, t in enumerate(a)))


def formula(a, scope, depth=0):
    return (type(a) is list
            and ((len(a) == 2 and all(term(t, scope, depth) for t in a))
                 or (len(a) == 1 and formula(a[0], scope, depth + 2))))


def opening(a, value, depth=0):
    if type(a) is int:
        return value if a == depth + 1 else a - (a > depth + 1)
    size = len(a)
    result = [opening(t, value, depth + (2 if size == 1 else int(size == 3 and i == 2)))
              for i, t in enumerate(a)]
    return result[1] if size == 3 and result[0] == 0 else result


def prove(a, p, g, n):
    require(formula(a, n))
    if len(a) == 2 and a[0] == a[1]:
        return
    match p:
        case ['v', c, b, h, t, major, minor]:
            require(len(a) == 2 and a[1] == opening(h, t))
            prove([b if c == 0 else [c, b, h], t], major, g, n)
            prove([[[0, c], b, h], a[0]], minor, g, n)
        case ['u', index, t, premise]:
            require(type(index) is int and 0 <= index < len(g) and term(t, n))
            antecedent, body = g[index]
            require(a == opening(body, t))
            prove(antecedent, premise, g, n)
        case ['e', body, major, branch]:
```

```
            prove([body], major, g, n)
            body = opening(body, -n - 1, 1)
            prove(a, branch, g + [(opening(body, 0), body)], n + 1)
        case ['i', t, branch]:
            require(len(a) == 1 and term(t, n))
            body = opening(a[0], t, 1)
            hypothesis = opening(body, -n - 1 if len(a[0]) == 2 and t == 0 else 0)
            prove(opening(body, [0, -n - 1]), branch, g + [([0, 0], hypothesis)], n + 1)
        case _:
            raise ValueError('invalid proof')


def check(c):
    require(type(c) is dict and set(c) == {'goal', 'proof'})
    prove(c['goal'], c['proof'], [], 0)


if __name__ == '__main__':
    try:
        check(json.load(sys.stdin))
    except Exception:
        print('REJECT')
        sys.exit(1)
    print('ACCEPT')
```

References

[1] V. Reshetnikov, *ProveIt: Concise presentations of intuitionistic primitive recursive arithmetic*, snapshot `1eb151b8f40800aec569361e827d934cb77bd3e1`, accessed November 11, 2026. https://github.com/VladimirReshetnikov/ProveIt/tree/1eb151b8f40800aec569361e827d934cb77bd3e1/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA.

[2] ProveIt incoming package, *Equality from iteration*, `ipra_equality_from_iteration_2026-11-11.zip`, in the snapshot of reference [1]. The zero-normal single evaluation rule and identity-frame argument are inherited from this package.

[3] ProveIt incoming package, *Five rules for iPRA with ordinary substitution*, `ipra_raw_clausal_2026-11-11.zip`, in the snapshot of reference [1]. The raw CE5 control and clause translation are inherited from this package.

[4] ProveIt, `Research/clausal_canonical/`, in the snapshot of reference [1]. Canonical clause presentation and its complete checker.

[5] ProveIt, `Lean/ConciseIPRA/ZeroNormalization.lean` and `NativeZeroNormal.lean`, in the snapshot of reference [1]. The endpoint `nzprf_iff_nprf` compares the native kernel with its zero-normal presentation.

[6] ProveIt, `Lean/ConciseIPRA/BiInterpretation.lean` and `NativeBiInterpretation.lean`, in the snapshot of reference [1]. Full native/T_I/iPRA interpretation chain.

[7] ProveIt, `Research/certified_zero/reference/`, in the snapshot of reference [1]. Inherited choice basis and ordinary arithmetic bridge, reproduced here from the attributed copy in the raw-clausal package.

[8] ProveIt contributors, *Internal scalar predecessor bootstrap*, `Research/certified_zero/reference/scalar_predecessor.tex`, in the snapshot of reference [1]; reproduced in this report.

[9] ProveIt contributors, *From literal-equation induction to full intuitionistic PRA*, `Research/certified_zero/reference/equality_reflection.tex`, in the snapshot of reference [1]; reproduced in this report.

[10] ProveIt contributors, *Pairing, primitive recursion, and internal numerical composites*, `Research/certified_zero/reference/primitive_recursion.tex`, in the snapshot of reference [1]; reproduced in this report.

[11] ProveIt, `Research/canonical_recursors/`, in the snapshot of reference [1]. Canonical syntax and the 516-byte compact checker reported for the separate codec.

[12] ProveIt, `Research/computational_equality/`, in the snapshot of reference [1]. Refl/Eval equality interface; formal comparison in `Lean/ConciseIPRA/NativeEval.lean`.